Product Requirements Document: Boardroom

Product	Boardroom, the Executable Whiteboard
Document type	Technical PRD and implementation reference
Status	Working prototype; deployed frontend at https://board-of-c.vercel.app
Code	`whiteboard-code-verifier/` in this repository
Audience	Engineers and reviewers with no prior knowledge of the implementation

---

1. Summary

Boardroom is a web whiteboard on which a teacher writes C++ or Python by hand or by keyboard, selects the program, and executes it for real. A Node.js API receives the request, reads handwriting with a locally hosted pretrained vision-language model (Qwen2-VL 2B), compiles and runs the code inside a disposable, network-isolated Docker container, and returns stdout, stderr, compiler and sanitizer diagnostics mapped back to source lines. Every run is archived as a Git commit containing the board, the exact source, the input and a reproducible replay bundle.

2. Problem

Code taught on a whiteboard is never executed. Students cannot tell whether an example works, teacher mistakes (a missing `&` in `scanf`, an off-by-one, a dangling pointer) go unnoticed, memory behaviour stays abstract, and moving the code into an IDE interrupts the lecture.

3. Goals and non-goals

Goals

1. Execute what is on the board without leaving it, with real compiler and runtime evidence.
2. Read handwriting with a pretrained model running locally, with no third-party AI API and no API key.
3. Never execute model output that a human has not confirmed.
4. Contain arbitrary submitted code so it cannot affect the host.
5. Persist every lesson as a reproducible, versioned record.
6. Keep the website public while restricting the compute-heavy API to the owner.

Non-goals

- Camera capture of physical whiteboards.
- Multi-user accounts, grading, exams or plagiarism detection.
- A public code-execution service for arbitrary internet traffic.
- Training or fine-tuning models.

4. Users

Persona	Need	Access
Teacher / site owner	Demonstrate and debug code live; keep lesson records	Full: holds the owner key
Student / visitor	Draw, download, and run typed code	Board and code running; OCR, snippets and lessons return HTTP 401 without the owner key

5. Functional requirements and user stories

ID	Story	Acceptance criteria	Implemented in
U1	Write or type code on a canvas and run it	Selecting a frame extracts its text; `POST /api/run` returns real stdout/stderr	`shared/board.js`, `server/index.mjs`, `server/execution/sandbox.mjs`, `runner/runner.py`
U2	Supply program input by writing in blue	Elements coloured `#1971c2` become stdin; other colours are ignored	`shared/board.js`, `shared/handwriting.js`
U3	Write by hand and have it read	Ink is exported as PNG, transcribed by Qwen2-VL, shown line by line; execution is blocked until the teacher confirms	`src/lib/capture.js`, `src/components/HandwritingPanel.jsx`, `server/ai/ocr.mjs`, `ocr-service/service.py`
U4	See errors where they occurred	Compiler, ASan and UBSan diagnostics carry line numbers and are underlined in the editor	`runner/runner.py`, `src/components/CodeEditor.jsx`
U5	Choose C++17 or Python 3	A language field selects compiler or interpreter, OCR prompt and snapshot file name	`shared/languages.js`
U6	Run several inputs	Up to 20 cases with expected status/output and first-mismatch reporting	`src/components/TestCases.jsx`, `src/lib/test-cases.js`
U7	Use teaching data structures	`stack.h`, `queue.h`, `linked_list.h`, `search.h` compile and report contract violations at the caller's line	`runner/lib/*.h`
U8	See what `malloc`/`free` did	Rule-based teaching notes for matching calls, ignoring comments and strings	`runner/runner.py`
U9	Keep a record of every lesson	Each completed run creates one Git commit with board, source, input and replay bundle; History lists the latest 30	`server/lessons/snapshots.mjs`, `server/lessons/github-snapshots.mjs`
U10	Replay an old lesson	A frozen runner, vendored headers and a SHA-256 manifest rebuild the run without the app	`server/lessons/snapshot-bundle.mjs`, `server/lessons/snapshot-replay.py`
U11	Write only loose C++ lines (Snippet mode)	The model adds boilerplate only; the app compares lines and warns on any change; the teacher confirms before execution	`ocr-service/service.py` (`/wrap`), `server/ai/snippet.mjs`, `src/main.jsx`
U12	Protect costly features while code running stays open	With `BOARDROOM_OWNER_KEY` set, OCR, snippets and lessons require the key; running code, health and public headers do not	`server/security/owner-key.mjs`, `server/hosted/hosted-api.mjs`

6. System architecture

```
Browser (React 18 + Excalidraw 0.18, built by Vite)
   |  HTTPS, JSON, Authorization: Bearer <owner key>
   v
Node.js 22 API  (server/index.mjs, port 8787, built-in http module, no framework)
   |-- owner-key check, origin/host check, body limits, rate limits
   |-- POST /api/ocr      -> OCR service  POST /ocr   (Python, Qwen2-VL, port 8788, loopback)
   |-- POST /api/snippet  -> OCR service  POST /wrap
   |-- POST /api/run      -> docker run whiteboard-runner:local  (gcc 14.2 + Python 3)
   |                         or Wandbox HTTP API when hosted on Vercel
   '-- /api/snapshots     -> local Git repository (snapshots/) or GitHub Git Data API
```

Component	Technology	Responsibility	Trust level
Frontend	React, Excalidraw, Vite, plain CSS	Board, selection, review, results	Untrusted (runs on visitors' machines)
API	Node.js built-ins only	Authentication, validation, rate limiting, routing	Trusted; holds secrets
OCR service	Python 3.12, PyTorch, Hugging Face Transformers	Handwriting transcription and snippet wrapping	Trusted; loopback only
Runner	Docker, gcc 14.2, Python 3 standard library	Compile and execute submitted code	Untrusted code inside an isolated container
Archive	Git (local) or GitHub	Lesson history	Trusted storage

Deployment modes

Mode	API	Execution	Archive	OCR
Local	`node server/index.mjs` on 127.0.0.1	Local Docker	`snapshots/` Git repo	Local service
VPS	Node behind Caddy (HTTPS)	Docker on the VPS	Disk Git, optional push	Service on the same host
Vercel	Serverless functions in `api/`	Wandbox	GitHub Git Data API	`OCR_SERVICE_URL` must point to a separate host

7. Data flows

7.1 Typed code

1. `shared/board.js` orders the frame's text elements by position, separates black (source) from blue (stdin), and builds a line-to-element map.
2. The browser sends `POST /api/run` with `{ source, stdin, language }`.
3. The API validates, checks limits, and calls `runSandbox()` in `server/execution/sandbox.mjs`.
4. The container runs `runner.py`, which writes `board.cpp` or `board.py`, compiles, executes with stdin on a pipe, and prints one JSON result.
5. The API returns the result; the browser maps diagnostic lines to the editor and the board.
6. The browser posts the scene to `/api/snapshots`, which writes and commits the lesson.

7.2 Handwriting

1. `shared/handwriting.js` detects pen strokes (`freedraw`, arrows) in the frame; `src/lib/capture.js` exports code ink and input ink as separate PNGs (up to 2048 x 2048).
2. `POST /api/ocr` validates the PNG header and size, strips the data-URL prefix and forwards base64 to the OCR service.
3. The service runs Qwen2-VL with a fixed transcription prompt and returns `{ code: [{ text, uncertain }], stdin: [...] }`. `uncertain` is set when the least-confident generated token has probability below 0.5.
4. The teacher edits and confirms the lines. A scene fingerprint invalidates the confirmation if the ink changes.
5. The confirmed text follows flow 7.1.

7.3 Snippet mode (C++)

1. The teacher enables **Snippet** and writes only loose statements.
2. `POST /api/snippet` sends `{ source, language: "c" }`; the service's `/wrap` asks the model to add only includes, `using namespace std;`, `int main() {`, `return 0;` and the closing brace, keeping function definitions outside `main`.
3. `checkKeepsLines()` in `server/ai/snippet.mjs` verifies every non-blank original line appears in the result, in order, ignoring indentation and repeated spaces. A mismatch yields a `warning` naming the first changed line.
4. The prepared program and any warning are displayed; it runs only after **Confirm & run**.

8. API specification

All bodies are JSON. With `BOARDROOM_OWNER_KEY` set, requests need `Authorization: Bearer <key>` except where marked public.

Method and path	Access	Request	Response	Limits
`GET /api/health ?language=c\	python`	Public	none	`{ runner, ocr, locked, authorized }`
`POST /api/run`	Public	`{ source, stdin?, language? }`	`{ status, stdout, stderr, compilerOutput, exitCode, durationMs, diagnostics[], notes[] }`	64 KiB body; 2 concurrent; 20/min per client, 120/min total
`POST /api/ocr`	Owner	`{ code?: dataURL, stdin?: dataURL, language? }`	`{ code[], stdin[], provider, model, durationMs, requiresConfirmation }`	4 MiB body; 1 concurrent; 6/min per client, 30/min total (shared with snippet)
`POST /api/snippet`	Owner	`{ source, language: "c" }`	`{ source, warning, provider, model, durationMs, requiresConfirmation }`	64 KiB body; shares OCR limits
`GET/POST /api/snapshots`	Owner	scene for POST	list, or `{ id, commit }`	4 MiB body

Method and path	Access	Request	Response	Limits
`GET /api/snapshots/<id>[/libraries]`	Owner	none	scene, or header comparison	none
`GET/POST /api/snapshots/sync`	Owner	none	archive sync state	none
`GET /api/library/<header>`	Public	none	header text (allowlisted names only)	none

`status` is one of `success`, `compile_error`, `runtime_error`, `timeout`, `output_limit`, `invalid_source`. Errors return `{ error, code?, diagnostic? }` with HTTP 400, 401, 403, 413, 415, 429, 502, 503 or 504.

9. Execution sandbox

`server/execution/sandbox.mjs` starts a new container per request:

```
docker run --rm --interactive --network none --read-only --cap-drop ALL
  --security-opt no-new-privileges --pids-limit 64 --memory 512m --memory-swap 512m
  --cpus 1 --ulimit core=0:0 --ulimit nofile=64:64 --ulimit fsize=16777216:16777216
  --tmpfs /tmp:rw,exec,nosuid,size=64m,mode=1777 --user 65534:65534 whiteboard-runner:local
```

Control	Value
Compiler flags	`-std=c++17 -g -O0 -Wall -Wextra -fno-omit-frame-pointer -fsanitize=address,undefined -fno-sanitize-recover=all`
Source / stdin / output	32 KiB / 8 KiB / 16 KiB
Compile / run deadline	10 s / 5 s wall (3 s soft, 4 s hard CPU)
Outer deadline	22 s, then the container is force-removed
Includes	Standard headers allowlisted in `runner.py` plus the four teaching headers

10. OCR service

Item	Specification
Model	`Qwen/Qwen2-VL-2B-Instruct` (Apache-2.0), loaded once with Transformers, float32 on CPU by default
Endpoints	`POST /ocr`, `POST /wrap`, `GET /health` on 127.0.0.1:8788
Image budget	Downscaled to at most 640 x 28 x 28 pixels (about 640 visual tokens)
Output budget	600 tokens for OCR, 1200 for wrapping; greedy decoding
Concurrency	One inference at a time (process lock)
Measured	One 5-line handwritten sample transcribed exactly in about 28 s on a laptop CPU; not a benchmark
Previous attempt	TrOCR (line-level) misread every line of the same sample and was replaced

11. Security requirements

Threat	Control	Location
Strangers using the costly API routes (OCR, snippets, lessons), including from devtools	Owner key compared as SHA-256 digests with `timingSafeEqual`; HTTP 401 otherwise	`server/security/owner-key.mjs`
Other websites calling the API	Host and Origin allow-lists, `Sec-Fetch-Site` check, CORS limited to the app origin	`server/index.mjs`, `server/hosted/hosted-api.mjs`
Flooding	Per-client and server-wide rate limits, concurrency caps, body limits, 10 s request timeout	`server/security/rate-limit.mjs`, `server/index.mjs`
Malicious submitted code	Docker isolation (section 9)	`server/execution/sandbox.mjs`
Secret leakage	Secrets only in server environment; never `VITE_`-prefixed, never mounted into containers, never logged	`.env.example`
Unreviewed model output running	Mandatory teacher confirmation for OCR and snippets; line-preservation warning	`HandwritingPanel.jsx`, `server/ai/snippet.mjs`
Privacy	Handwriting processed locally; no accounts, database or analytics; logs omit images and text	`ocr-service/service.py`
Tampered lesson replays	SHA-256 manifest verified before rebuild	`server/lessons/snapshot-bundle.mjs`

12. Configuration

Variable	Purpose	Default
`BOARDROOM_OWNER_KEY`	Makes the API private to the key holder	unset (open)
`BOARDROOM_APP_ORIGIN`	Allowed frontend origin for a separate API host	unset
`BOARDROOM_TRUST_PROXY`	Use `X-Forwarded-For` for rate limits behind a proxy	off
`OCR_SERVICE_URL`	OCR service address	`http://127.0.0.1:8788` (required on Vercel)
`OCR_MODEL`	Model checkpoint	`Qwen/Qwen2-VL-2B-Instruct`
`OCR_TIMEOUT_MS`	API wait for the OCR service	180000 local, 50000 Vercel
`SNAPSHOT_DIR`	Local archive location	`snapshots/`
`SNAPSHOT_GITHUB_REPOSITORY`, `_BRANCH`, `_TOKEN`	Hosted archive	unset
`WANDBOX_API_URL`, `WANDBOX_CPP_COMPILER`, `WANDBOX_PYTHON_COMPILER`	Hosted execution	unset

13. Non-functional requirements

Category	Requirement
Performance	Typed code: result within the 22 s outer limit. OCR or snippet: within `OCR_TIMEOUT_MS` (about 10-30 s on CPU)
Resources	OCR service: about 4.5 GB disk and 10 GB RAM; runner: 512 MiB per container
Reliability	Containers removed after timeouts; snapshot Git failures keep files and warn
Maintainability	Runtime dependencies limited to React, Excalidraw and the Python ML stack; Node API uses built-ins only
Testability	Node `node:test` and Python `unittest` unit tests; Playwright browser tests; real-Docker smoke checks

14. Testing

Suite	Command	Scope
Unit	`npm test`	Extraction, OCR and snippet clients, rate limits, owner key, API handler, snapshots, runner fixtures
Container	`npm run test:docker`	Lessons, EOF, timeout, output limit, leak detection in the real image
Browser	`npm run test:browser`	UI flows in Chromium with stubbed OCR and real Docker
OCR smoke	`npm run test:ocr -- image.png`	One PNG through the running OCR service

Known: snapshot-replay tests fail on Windows hosts (bash and file-locking differences); they pass on Linux.

15. Risks and mitigations

Risk	Mitigation
Model misreads handwriting	Mandatory review, uncertainty flags, typed fallback
Model "fixes" teacher mistakes in OCR or snippets	Prompts forbid repairs; snippet line check warns; teacher confirms
Slow inference on CPU	Visual-token cap, long timeouts, single-job lock; GPU recommended
Weak or leaked owner key	Use 20+ random characters; key never in the build; rotate by changing the variable
Hosted execution is third-party	Documented as weaker than Docker; Docker mode recommended

16. Success metrics

- Handwriting to a running result in under one minute for a short program.
- 100% of handwritten or snippet-prepared programs confirmed by a human before execution.
- Compiler errors point to the correct line for typed code.
- With the owner key configured, 0 API operations succeed without it.
- A saved lesson replays from its Git record alone.

17. Roadmap

Horizon	Items
Next	Benchmark OCR on classroom samples; per-user accounts; rate-limit failed key attempts; "lock" button; GPU deployment for the model
Later	Fine-tune the model on code handwriting; stronger sandbox (gVisor or Firecracker); student view and shared sessions; more languages

18. Repository map

Path	Contents
`whiteboard-code-verifier/src/`	React frontend (`main.jsx` is the application shell)
`whiteboard-code-verifier/server/`	Node API and its modules
`whiteboard-code-verifier/api/`	Vercel function wrappers
`whiteboard-code-verifier/runner/`	Container image, `runner.py`, teaching headers
`whiteboard-code-verifier/ocr-service/`	Python model service
`whiteboard-code-verifier/tests/`	Unit and browser tests
`docs/`	This PRD and the plain-language project report